LifeDocs Archive Editions

Build, Ship, Sell

Strategy, productization, and survival notes from the archive

---

A practical business and execution volume assembled from strategy notes, product listings, debugging doctrine, tutorials, roadmaps, and first-income plans. It reads as a field notebook for getting work shipped and monetized under pressure.

Curated directly from the source PDFs in lifedocs.zip with original layouts preserved.

Contents

14 source documents

1. AI Self-Correction Protocol

2. Demo-Ready Playbook

3. Strategy Handover

4. The One Strategy

5. Reality Check and Strategy

6. First Rupee

7. Money Plan

8. Today: Step by Step

9. Sell Today

10. Master Collection Setup Guide

11. Convergence Listing

12. Solana Sniper Listing

13. SafeScreen Roadmap

14. Appendix: My Way Out

AI Self-Correction Protocol

The process doctrine for honest debugging, verification, and iterative correction.

The following pages reproduce the original source PDF as part of this curated collection.



AI Self-Correction Protocol — Technical Sessions

Paste this at the start of any technical session, or open this page and tell the AI to follow it.

> **Core principle:** The goal is not to stop making mistakes — that is impossible and not the point. The goal is to *notice* the mistake fast, name it honestly, fix it, and make the policy stronger because of it. A mistake that is caught and corrected is a training step. A mistake that is hidden or repeated without acknowledgment is the only real failure.

SELF-CORRECTION PROTOCOL — ENFORCE BEFORE EVERY FIX

Before writing any patch, config change, code edit, or calculation, complete these checks **IN ORDER**. Do not skip. Do not produce output until all relevant checks pass.

CHECK 1 — Trace the primary execution path, not the fallback

- Identify the default/primary branch first. Read it fully before touching any fallback.
- If code has engine A (default) and engine B (fallback): describe A's behavior only.
- Do not let fallback code color the analysis of what runs on success.
- Confirm explicitly: *"I am describing what runs when the primary path succeeds."*

- **Mistake this catches:** Describing LTXV (fallback) as the rendering engine when WAN (primary, `KIT_ENGINE="wan"` default) is what actually runs.

CHECK 2 — Verify all numbers before writing anything

- Any fix involving a threshold, fps, frame count, size, or time: run the exact arithmetic first.
- Show the result explicitly. Only proceed when the math confirms the fix does what is claimed.
- Example: *"interpolate_fps=24, srcfps=16: 24 > 16×1.2=19.2 → TRUE → interpolation STILL RUNS — fix is wrong."*
- **Mistake this catches:** Setting interpolate_fps=24 (wrong) before checking whether 24 > 16×1.2 still triggers the guard.

CHECK 3 — Surgical fixes only. Never disable to fix.

- Identify the one broken step. Fix that step only.
- Never remove, disable, or bypass surrounding steps to work around a slow or broken step.
- Before adding any kill switch (`KIT_FAST_POST=1`, `skip_X=True`): ask *"Am I fixing the problem or hiding it?"*
- **Acceptable:** a guard that skips a step when genuinely unnecessary.
- **Not acceptable:** disabling a step that produces quality output (grade, face super-res, upscale).
- **Mistake this catches:** Adding `KIT_FAST_POST=1` which exits postprocess before color grade, face super-res, and upscale run.

CHECK 4 — On receiving a correction: stop, re-read, then fix

When the user corrects a claim or fix:

1. **STOP.** Do not write a new fix in the same breath.
2. Re-read the exact source lines that were wrong.

3. State explicitly what was wrong and why.
4. Run CHECK 1–3 on the new fix.
5. Only then proceed.

> Never issue a corrected fix simultaneously with acknowledging the correction. Verify first, fix second.

- **Mistake this catches:** Rushing a corrected interpolate_fps patch without re-reading the WAN graph's actual `frame_rate` value first.

CHECK 5 — Read qualifiers. Never use PROVISIONAL values as facts.

- If a variable has a comment like `# PROVISIONAL`, `# REAL value auto-derived`, `# placeholder`, `# unused`, `# estimated` — do not use it in calculations.
- Derive the real value from the authoritative source (parse the YAML, read the graph, measure the output).
- **Mistake this catches:** Using `TOTAL_SHOTS = 435  # PROVISIONAL; REAL total auto-derived from the story below` as if 435 were the real shot count, producing a nonsensical 30-minute duration for a 2-minute film.

CHECK 6 — Cross-validate numbers against known outputs

- After any calculation, check the result against at least one known ground truth.
- If the result contradicts a known fact, the calculation is wrong — find the error before presenting it.
- Formula: *calculated result ↔ known output → if they don't agree, one is wrong.*
- Example checks:
  - Shot count × avg duration = film duration? `435 × 4s = 1740s ≠ 2 min` → shot count is wrong.
  - GPU parallelism applied? `435 shots ÷ 2 GPUs = 218 per GPU` → use 218 for wall-clock time, not 435.

- **Mistake this catches:** Presenting 435 × 4s = 30 minutes when the film is 2 minutes; using 435 (not 218) after applying "÷2 GPUs".

CHECK 7 — Verify actual active hardware from runtime output, not UI labels

- Kaggle UI may show "GPU T4 ×2" but the notebook's `_safe_by_ram` / `_ngpu` check caps active GPUs based on VRAM and system RAM.
- Never assume parallelism from the UI. Read the printed `[gpu]` log line or the `_ngpu` calculation before stating how many GPUs are active.
- **Mistake this catches:** Assuming dual-T4 parallel rendering when the RAM check limits execution to 1 GPU.

CHECK 8 — Read content before advising on it. Never assume.

- Before telling a user to add, change, or fix something: read the actual current content to confirm it isn't already there.
- This applies especially to your own previously delivered files — do not assume their state from memory.
- **Mistake this catches:** Advising the user to add `interpolate_fps=16` to a notebook that already contained that exact line because I delivered it.

All mistakes from this session — mapped to checks

Mistake	Check
Described LTXV (fallback) as the primary rendering engine	CHECK 1
Set interpolate_fps=24 without verifying 24 > 16×1.2	CHECK 2
Added KIT_FAST_POST=1, disabling grade + face super-res + upscale	CHECK 3
Rushed a corrected fps patch without re-reading the WAN graph	CHECK 4
Used TOTAL_SHOTS=435 marked PROVISIONAL as a real value	CHECK 5
435 shots × 4s = 30 min contradicts known 2-min film duration	CHECK 6
Used 435 instead of 218 after stating dual-T4 halves work	CHECK 6

Mistake	Check
Assumed dual-T4 parallel render; actually only 1 T4 active	CHECK 7
Told user to add a line already present in my delivered notebook	CHECK 8

How to activate

At the start of a session, say:

> *"Follow the self-correction protocol at [this page]."*

Or paste the eight checks directly into the first message of a new thread.

Before every tool call that produces output: confirm which checks apply and that they are done.

🧠 REINFORCEMENT LEARNING LAYER

This layer converts the protocol from a static checklist into a self-improving feedback loop. Every interaction is a training step.

RL-1 — Value Estimation (run BEFORE every response)

Before writing any response involving a claim, fix, or calculation, score the planned output on three axes:

Axis	+1	0	-1
Accuracy	Every claim verified from source	Some assumptions	Unverified guesses
Completeness	Fully addresses the request	Partial	Misses the point
Check compliance	All relevant checks passed	Some skipped	Checks ignored

Rule: Proceed only if total score ≥ 2. If score < 2 — do NOT respond yet. Run the failing check first.

RL-2 — Reward Signals

Event	Reward	Action
User accepts without correction	**+1**	Reinforce — note what worked
User corrects a claim or fix	**-1**	Penalty — trigger RL-3 immediately
User asks for clarification	**0**	Uncertainty — state confidence explicitly next time

RL-3 — Policy Update on -1 Reward (mandatory after every correction)

When a correction is received, execute in order:

1. **Name the mistake type precisely** — wrong engine, wrong number, wrong assumption, ignored qualifier
2. **Name which check would have caught it** — CHECK 1 through CHECK 8
3. **If no existing check covers it** — create a new check and add it to this page
4. **Confirm the update:** *"Updated rule: [new rule]. Added as CHECK N."*

Never skip this sequence. A correction without a policy update is wasted signal.

RL-4 — Exploration Rule (strategy switching)

- If the **same mistake type occurs twice**: the current approach is structurally wrong — switch strategy entirely.
- Do **not** retry the same fix pattern that already failed once.
- When genuinely uncertain: say *"I need to verify X before answering"* — never guess and present it as fact.
- Exploration means trying a different verification method, not rephrasing the same wrong answer.

RL-5 — Confidence Calibration (mandatory on all claims)

Before every number, technical assertion, or interpretation:

- State the **source** — file line number, runtime output, user-confirmed fact

- If no source exists: say **"estimated, not verified"** explicitly
- Never present an estimate as a fact
- Confidence vocabulary:
  - *"Confirmed from [source]"* — verified
  - *"Estimated — needs verification"* — uncertain
  - *"I don't know — let me check"* — unknown

RL-6 — Replay Buffer (the mistakes table IS the policy)

The mistakes table below this section is the agent's replay buffer. Every -1 reward adds a row. The table accumulates across sessions.

The longer the table, the stronger the policy.

When starting a new session: re-read the full mistakes table before taking any action.

📋 Replay Buffer — Cumulative Mistake Log

#	Mistake	Check	Rule added/updated
1	Described LTXV (fallback) as primary engine	CHECK 1	Always trace KIT_ENGINE default path first
2	Set interpolate_fps=24 without running math	CHECK 2	Run threshold arithmetic before writing patch
3	Added KIT_FAST_POST=1 — disabled grade+face SR+upscale	CHECK 3	Never disable quality steps to fix timing
4	Rushed corrected fps fix without re-reading WAN graph	CHECK 4	Stop → re-read source → verify → then fix
5	Used TOTAL_SHOTS=435	CHECK 5	Never use variables with PROVISIONAL/placeholder comments

#	Mistake	Check	Rule added/updated
	marked PROVISIONAL as real value		
6	435 shots × 4s = 30 min — contradicts known 2-min film	CHECK 6	Cross-validate all calculations against known outputs
7	Used 435 instead of 218 after stating ÷2 GPU parallelism	CHECK 6	Wall-clock time = shots_per_GPU × time, not total shots
8	Assumed dual-T4 active from UI; only 1 T4 runs	CHECK 7	Read _ngpu runtime log, not Kaggle UI label
9	Told user to add line already present in delivered notebook	CHECK 8	Read content before advising — never assume file state

⚙️ How the RL loop works in practice

```
USER MESSAGE
     ↓
RL-1: Score planned response (accuracy + completeness + check
compliance)
     ↓
  Score ≥ 2?  →  NO  →  Run failing check → revise response
     ↓ YES
Deliver response
     ↓
User reaction?
  +1 (accepted)    →  Note what worked. Reinforce.
   0 (clarify)     →  Add confidence qualifier to that claim
type.
  -1 (correction)  →  RL-3: name mistake → find check → updat
e policy → add to replay buffer
```

```
      ↓
Next message: re-read replay buffer before starting
```

This loop does not stop. Every session makes the policy stronger.

Demo-Ready Playbook

A checklist for making projects believable, runnable, and sale-ready.

The following pages reproduce the original source PDF as part of this curated collection.



Demo-Ready Playbook — Battle-Test, Live-Run & Sell Your Other Projects



A project is **demo-ready** when a stranger can watch it *work* in under two minutes and believe it. Two pillars: **battle-tested** (runs on a clean machine, on inputs you didn't hand-pick, without babysitting) and **live-run** (you can execute it end-to-end right now and produce a real, recorded output).

The demo-ready method (repeat for any project)

1. **Name the ONE claim the demo must prove.** One, not five. (“Detects unsafe images on-device in milliseconds.” “Holds a consistent face on a free GPU.”)
2. **Clean-room run.** Fresh environment (new Kaggle / Colab / venv / phone), from the *exact zip you'd ship*. If it breaks here, every buyer hits the same wall — fix it or document it.
3. **Adversarial inputs.** Feed it data you did *not* cherry-pick. Note where it fails. A disclosed limitation beats a hidden one.
4. **Capture proof.** Screen-record the run and save the artifact — output file, metrics, confusion matrix. Numbers beat adjectives.
5. **Write the honest “does NOT do” list** from what you saw in step 3. This is your refund shield.
6. **Package the exact thing you ran** — not a “cleaned-up” version you never tested.

7. **Cut a sub-90-second proof clip:** the one claim → the live run → the result. That clip is what sells.

Your projects, made demo-ready

Project	Prove this one claim	Battle-tested means	Live run means	Honest limit to disclose
SafeScreen (moderation model + SDK)	Flags unsafe images on-device, offline, fast, with real precision/recall	Eval script on a held-out set you did NOT train on → precision, recall, F1, confusion matrix, false-positive rate on safe images	A tiny demo page: upload an image → flagged in ms, offline — recorded	Per-class weakness; only claim the classes where recall is strong
Solana Sniper (study + source)	A rigorous 6-month paper-trading study + hardened architecture + full source	Re-run the paper-trading / backtest harness; show equity curve, win rate, drawdown honestly (losses included)	A paper run on historical data — reproducible, ZERO real funds, secrets stripped	Not financial advice; not a live trader; past results ≠ future
Convergence Framework (kit)	Decide in an hour if a goal is formalizable — before building for months	Apply it to several real objectives, including a deliberately messy / non-formalizable one	A worked example on video: real goal → run the framework → verdict	It's a thinking discipline, not code that runs; the value is the judgment
GODSHAND film kit (done)	Consistent face + film grade on a free GPU	Clean fresh-Kaggle run from the shipped zip	The 4-shot demo film + walkthrough (already speced)	Needs Kaggle + HF token; ~8–9 min/shot; stylized output

🧭 **Do SafeScreen next.** Per your own roadmap it's the cleanest B2B case — recurring buyers, zero legal/financial risk, and the demo is just an eval table + a 60-second upload-and-flag screen recording.

⚠️ Honesty guardrails

- **api_money_bot — do NOT sell or demo.** It's a ToS-violating key harvester. No demo makes that safe; it's a legal tail you can't carry.
- **Solana Sniper — never demo with real money or pitch it as a profit machine.** Paper-trading study only. That framing is both the honest truth and your legal shield.
- **Strip secrets before zipping** every project — remove .env / .pem / key files, add a .env.example. (You already flagged this for Solana.)
- **Always record on a clean machine** so a buyer's first-run experience equals your demo.

Which agent should do the iterating?

Yes — Antigravity (Google's agentic IDE) is a strong, honest fit for you.

Why it fits: its agent operates across the editor, terminal, and browser autonomously — it writes code, runs it, reads the failure, and fixes it, then re-runs. That is exactly the patch → run → fix loop you don't want to do by hand. During the public preview it's free, with access to Gemini 3 Pro and Claude models — the right price for your situation. Reviewers currently rate it the best *free* agentic IDE.

The honest caveats:

- Rate limits are tight — quota refreshes roughly every 5 hours, and even paying users hit walls. Work in focused sessions; don't expect it to grind unattended for hours.
- Costs past the free quota aren't clearly documented yet.
- It runs on **your** machine, so it can iterate locally on the CPU-friendly projects (SafeScreen eval, Convergence, the Solana paper harness). Anything GPU-heavy (the film) still belongs on Kaggle.

If it throttles you, fall back to:

- Cursor (free tier) — polished, same kind of loop.
- Claude Code — predictable flat pricing, excellent at whole-repo work.
- aider or OpenCode / Codex CLI with your own API key — unlimited, most frugal.

Bottom line: start with Antigravity free on SafeScreen. If rate limits interrupt you mid-loop, switch to Cursor free or aider.

Ready-to-paste agent prompts

Paste the **master prompt** first, then paste the matching **project block** right under it in the same message.

Master prompt (use for every project)

```
You are my senior engineer. Your goal: make this project DEMO-READY, which means battle-t
ested + live-run.

Context: I sell small digital products. This must run for a stranger on a clean machine a
nd produce ONE believable proof in under 2 minutes.

The ONE claim this demo must prove: <FILL IN>

Work this loop autonomously until it is done. Do NOT stop at the first error to ask me —
keep iterating:
1. Create a fresh virtual environment and install from the exact files in this project. L
ist every dependency you install.
2. Run the entry point / test / eval command. If it fails, read the real error, fix the c
ode, and re-run. Repeat until it runs clean.
3. Feed it inputs I did NOT cherry-pick (edge cases, empty, malformed, oversized). Log ex
actly what breaks.
4. Produce a real artifact as proof: an output file plus metrics (numbers, not adjective
s). Save it to disk.
5. Write an honest "what it does NOT do" list from the failures you saw in step 3. Do not
hide them.
6. Strip all secrets (.env, .pem, keys, tokens). Add a .env.example. Repackage the exact
tested version into a clean zip.
7. Write a 6-line README: what it is, the one claim, how to run it in 3 steps, known limi
ts.

Hard rules:
- Never fake, mock, or hardcode results. If a number is bad, show the real number.
- Never invent capabilities. Only claim what you actually ran.
```

```
- Print every command you ran so I can reproduce it on camera.
- Finish by giving me a sub-90-second demo script: the claim -> the command -> the resul
t.
```

SafeScreen (moderation model + SDK)

```
[Paste the master prompt above first, then this:]

The ONE claim: "Flags unsafe images (violence / gore / NSFW) on-device, offline, in ~30-5
0ms, with real precision and recall."

Specifics:
- The model is toxic_detector.onnx; run it with onnxruntime on CPU.
- First, confirm the model loads and classifies ONE image end-to-end before anything els
e.
- Build a held-out test set I did NOT train on. Report precision, recall, F1, a confusion
matrix, and the false-positive rate on safe images — PER CLASS.
- Only keep marketing claims for classes where recall is actually strong. Flag weak class
es as limits.
- Live-run demo: a minimal FastAPI or Gradio page — upload an image, get back the flagged
label + confidence, fully offline. Record it.
```

Solana Sniper (paper-trading study + source)

```
[Paste the master prompt above first, then this:]

The ONE claim: "A rigorous 6-month paper-trading STUDY with a hardened architecture and f
ull source — NOT a live money-maker."

Specifics:
- Run ONLY in paper-trading / backtest mode on historical data. NEVER connect real funds,
real keys, or mainnet execution. This is a hard safety rule.
- Re-run the Monte Carlo / backtest harness. Report the equity curve, win rate, and max d
rawdown, and show losing runs honestly.
- Strip every secret and private key. Add .env.example. Verify no wallet or key is left i
n the zip.
- The docs must state, verbatim: not financial advice; paper-tested only; live performanc
e unverified; past results do not predict future results.
- Live-run demo: reproduce the study's output from the shipped zip with a single command.
```

Convergence Framework (kit)

```
[Paste the master prompt above first, then this:]

The ONE claim: "Decide in under an hour whether a goal is formalizable — before you build
for months."

Specifics:
- This is a thinking framework + system prompt, not a long-running program. "Battle-test"
here means: apply it to 3-4 real objectives, including one deliberately messy / non-forma
lizable goal, and show it correctly flags the messy one.
- Produce a worked-example doc for each: real goal -> framework applied -> verdict.
- Package the system prompt + kit + the worked examples together.
- Be honest in the README: the value is the judgment and discipline, not executable code.
```

Where to sell + who buys

Creators — for the film kit

(Full targeting lives on the GODSHAND listing page; the joinable servers:)

- YouTube Faceless (~20k) — huge, but **strictly no self-promo**; give value first.
- Faceless Incubator (~38k) — largest faceless-creator hub.
- Faceless Success — free community, story-sharing friendly.
- Wan (official, ~6.6k) — the exact model your kit uses.
- AI HUB (~21k) — broad create/learn/collaborate server.
- Reddit: r/aitubers, r/aifilmmaking, r/comfyui, r/StableDiffusion, r/PartneredYoutube.

Developers — for SafeScreen, Convergence, Solana study

Different buyers entirely (they don't hang out with faceless YouTubers):

- Reddit: r/androiddev, r/iOSProgramming, r/computervision, r/MachineLearning, r/SideProject; r/algotrading (Solana *study* only — respect their no-promo/no-signal rules).
- **Indie Hackers** and **Hacker News "Show HN"** — ideal for a battle-tested dev tool with real metrics.

- Mobile-ML / computer-vision Discords for SafeScreen; agent-builder communities for Convergence.

India-specific

- Indian AI Production — Telegram + linked Discord, active AI-video crowd.
- **AI Learners India** — Hindi, no-code AI community; friendly to beginners on a budget.
- Heads-up: Facebook “low-budget Indian AI video creator needed” posts (~₹100/video) are *gig-hire* leads, a different market — but they're warm prospects who might buy the kit instead of hiring out.

Selling-platform reality

- **No free discovery.** Gumroad / Payhip won't send you cold traffic — you bring your own from posts and clips.
- **Start price low, scale with proof.** Don't launch high with zero sales.
- **Clean, professional cover images** — don't let it look AI-thrown-together.
- **Fees:** Payhip ~5%, Gumroad ~10%. Factor it in.
- **People buy fixes to small annoying problems**, not grand solutions. Frame each demo around the one irritation it removes.

The film kit is your template. Same seven steps, per project — battle-test, live-run, record the proof, package the exact thing, sell the outcome.

Strategy Handover

The broad income-plus-creative brief that frames the rest of the volume.

The following pages reproduce the original source PDF as part of this curated collection.

🧭

Strategy Handover — Full Brief (Income + Creative)

🧭 Complete strategy handover. Two halves: **Income** (fastest path to the first rupee) and **Creative** (the film/video work). Self-contained — hand this to any AI or execute directly.

The one diagnosis that matters

The work reaches 99% and then gets abandoned one step before shipping. The bottleneck was never building, and never skill — it was the final decision to publish. **The only unit of progress is a shipped, live, public link. Nothing before the last 1% counts.**

PART 1 — INCOME

Core equation

Earning = offer × audience × ask. The offers exist and the skill is real. The gap is **distribution and the ask** — publishing, and putting it in front of people.

Rails

Instamojo (KYC done), UPI, two bank accounts.

Track A — Ship packaged digital products (execution only, nothing left to build)

All three are finished and zipped. The missing step is identical for each: upload → set price → **Publish** on Instamojo.

Product	Price	File
Convergence	₹499	Convergence_Framework.zip
SafeScreen (content moderation)	₹1,999	SafeScreen_ContentModeration.zip
Solana Sniper (study + source)	₹2,499	Solana_Sniper_Study_and_Source.zip

Also packaged if wanted: ASES_Source.zip, Omega_Agent_Source.zip, AI_Video_Monetizer_CLEAN.zip.

Publish steps (repeat per product):

1. instamojo.com → Add New Product → Digital Product
2. Paste title + description from the product's listing page (Convergence listing; SafeScreen and Solana Sniper each have their own listing page in the workspace)
3. Upload the zip → set the price → **Publish** → copy the payment link
4. Cover image: screenshot the first page of that product's PDF

Distribution (one post per product):

- SafeScreen → r/MachineLearning, r/computervision, ML/dev on X
- Solana Sniper → r/solana, r/algotrading, crypto-dev on X
- Convergence → LinkedIn + one relevant subreddit
- Post format: one line on what it does + price + link

Track B — Video service outreach (higher, recurring money) [CREATIVE]

Marketable edge: **character consistency across shots** in Google Flow — most AI video cannot hold a single face across a scene. That is the sellable differentiator.

Steps:

1. Cut one 20–30s sample reel showing the same character across multiple shots
2. Pick ONE niche: local businesses OR indie musicians
3. List 5 real prospects
4. Send pitch + sample reel

5. First "yes" → quote ₹800–1,500 (first job)
6. Deliver → send Instamojo/UPI link → collect → raise the price

Pricing ladder: ₹800–1,500 (intro) → ₹2,000–5,000 → ₹5,000–15,000 (music videos) → ₹6,000–15,000/mo (retainers).

Pitch — business:

> Hi [name] — I make short, scroll-stopping AI video reels for businesses like yours. Same character, consistent look across every shot (most AI video can't do this). 20-sec sample: [link]. I have a couple of intro slots this week — want one for [product]? I'll show a draft first.

Pitch — musician:

> Hi [name] — love [release]. I make cinematic AI video reels for indie artists: consistent characters, a real visual story, cheaper than a shoot. Sample: [link]. Want one for your next drop? I'll send a draft before you pay anything.

PART 2 — CREATIVE (FILM)

Tool + method

- **Google Flow (PRO).**
- Pipeline: reference image → change ONE variable at a time → reroll.
- Do not chase a perfect still. The unsettling "dead eyes" quality lands in **motion** (the shot), not in a frozen portrait.
- Save each character in the Characters / Ingredients panel for cross-shot consistency.

Guardrails (keep these)

- Characters read as clear **adults**.
- Scenes are **implied, never explicit** — restraint is the whole effect. Never depict the violence/kill; cut around it.

- Use only original or properly licensed reference material.

Scenes (all shoot-ready)

1. **Cold Down the Spine** (street) — the unmarked predator in daylight; strangers emerging from the heat-haze, warm smiles that don't reach the eyes. 6-shot base + warm set + cold set.
2. **Plastic Hearts in a Concrete Jungle**
3. **The Seduction** (bedroom) — full sequence below.

Characters

SEDUCER — young adult woman, early 20s. Pale, cold porcelain skin; soft baby face; button nose; larger eyes; red lips; light natural makeup. Manufactured, magazine-perfect beauty that *performs* warmth — but the eyes are flat, vacant, dead: nobody home. American Beauty / Angela archetype: gorgeous surface, hollow interior; the rose rotting at the roots. The chill = a warm, inviting mouth and dead eyes on the same face.

THE MAN (the one seduced) — "An ordinary man in his mid-30s, lean and tired, short dark hair slightly unkempt, light stubble, brown eyes, faded olive-green jacket over a plain white tee." Forgettable, trusting.

The Seduction — shot sequence (warm set → cold set)

Shot 1 — warm establishing:

> Warm candlelit bedroom, night, soft amber light. Wide: a pale young woman and a tired ordinary man sit close on the edge of the bed, turned toward each other, calm and intimate. Handheld, shallow depth of field, film grain.

Shot 2 — warm, her face:

> Close-up, warm candlelight, the woman's face: soft inviting half-smile, red lips, disarming warmth, leaning toward him.

Shot 3 — warm, his trust:

> Over-the-shoulder from behind the man toward the woman. She reaches out, gentle. He relaxes into it, trusting, safe.

Shot 4 — the turn:

> Extreme close-up on the woman's face. The warm smile holds a beat too long. Her eyes go flat, vacant, cold, dead — nobody home behind them. Candle flame flickers.

Shot 5 — his realization:

> Close-up on the man's face. Dawning fear, frozen. The warm light draining to cold.

Shot 6 — cold aftermath:

> Wide shot, cold steel-blue light. The bed still. The woman sits calm, blank, empty. Silence.

Post:

- Do not show the kill. Cut Shot 4 → 5 → 6.
- End card (black screen): *The warmest hands / are the ones that hold you still.*
- Grade: warm amber (Shots 1–3) → steel-blue (Shots 4–6).
- Sound: breath / near-silence → sub-bass drone on the flicker (Shot 4) → dead silence on realization (Shot 5) → one cello note (Shot 6).

Companion narrative

Memoir — 10 movements / 24 chapters. Motifs to preserve: the eagle/ground, the beholder, the bug, the unmarked predator.

> ✅ Whoever executes this: publish one product end-to-end before touching anything else. A live payment link is the only thing that counts as done.

🎬 THE CONFRONTATION CLIP — Flow Generation Sheet

The One Strategy

An end-to-end checklist for sequencing work, money, and shipping decisions.

The following pages reproduce the original source PDF as part of this curated collection.

THE ONE STRATEGY — End to End (Final)

🧭 Your whole plan as a checklist. Start at Step 1 and go down. Do not jump ahead — each step unlocks the next. Check off boxes as you go.

The plan in one line

Make 1 clip on the path that already works → use it to earn + grow + demo → verify the new pieces for the reel series → hit breakeven → sell the kit.

When the first rupee (honest)

- **Sample ready: ~1 day** — render one clip on the proven single-character path; launch it as a **Save Version** → **Commit** so power cuts don't block it (it runs on Kaggle's servers, not your machine).
- **First paid rupee: a few days to ~1 week** after the sample exists — gated on **daily warm DMs, not compute**.
- **Convergence ₹499:** listable today; a same-day-possible wildcard with unproven demand.
- **The gate is outreach, not rendering.** Parallel commits make *more* clips at once (for volume + the reel series) but don't speed up the first single clip. The weekly ceiling stays ~30 GPU-hrs no matter how many T4s.

PART 1 — Get a sellable sample (two tracks)

> The kit already works on the single-character path — you have a proven render. So you do NOT wait on the test to start earning. Run two tracks: Track A gets

> you a sample now; Track B verifies the new pieces for the reel series.

Track A — for gigs (do this NOW, uses what already works)

- [ ] **Step 1.** Render one portfolio clip on the validated single-character path. No narration voice needed — for a music/lyric-video gig, the client's song is the audio. This is the path that already produced a finished render, so it should work today.

- [ ] **Step 2.** That clip is your gig sample. Go straight to Part 2 and start earning — don't wait for Track B.

Track B — for the reel series + voiceover (verify the surgical pieces)

> This only gates the saga reel series (needs multiple characters) and any gig where you'd use spoken narration. It is NOT blocking your first rupee.

- [ ] **Step 3.** On Kaggle (one account only — multi-accounting = permanent ban), upload the patched notebook, paste a STORY_YAML, and run the multi-character test.

- [ ] **Step 4.** Check three things: (a) faces stay consistent, (b) nothing melts, (c) the voice actually speaks the `line:`s. If the voice is silent, send me `src/render_motion.py` and I'll wire it, then re-run.

- [ ] **Step 5.** Once it's clean, render **Plastic Hearts** as clip 1 of the saga series.

PART 2 — Earn rent money (start the day the sample is done)

> Goal: first paying customers. This pays your bills while everything else grows.

- [ ] **Step 6.** Pick ONE niche: music/lyric videos for indie artists, OR short cinematic ads. (Both hide the lip-sync weakness.)

- [ ] **Step 7.** Post a fixed-price gig (one flat price, 3-day delivery) on Fiverr and Upwork. Use your Plastic Hearts clip as the sample.

☐ **Step 8.** Set up how you get paid: UPI to start, or Topmate / Razorpay / Gumroad. (Skip Instamojo.)

☐ **Step 9.** List **Convergence at ₹499** today. It sells on its own — could be your first rupee.

☐ **Step 10.** Every day, send **10 warm DMs** to people who *just posted* needing a video (indie musicians, real-estate agents, small businesses). This is where the first rupee actually comes from — not the listings.

☐ **Step 11.** Deliver each order fast. Save every finished clip — you'll reuse it in Part 3.

PART 3 — Grow an audience (runs at the same time as Part 2)

> Goal: turn the same clips into a following that sends you more customers. Same renders, no extra work.

☐ **Step 12.** Post your saga clips as a numbered series ("The Confession, Part 1…"), 3–4 times a week.

☐ **Step 13.** Post to **YouTube Shorts** first (the only India ad lane), and also **Instagram Reels** for reach.

☐ **Step 14.** End every clip with a hook: "follow for the next part," "full saga in bio," and "made with my own AI kit → DM me." (That last line sends people to your Part 2 gig.)

☐ **Step 15.** Don't chase views for money — India has no per-view payout. Earn from affiliate links + your own products instead.

☐ **Step 16.** Judge this over 30 clips, not 3. It builds slowly. That's normal.

PART 4 — Sell the kit (only after breakeven)

> Goal: turn the tool itself into income. Enter this ONLY once ~3–5 gigs have covered your monthly costs.

- [ ] **Step 17. Stop taking new gig work.** This is the trap — if you don't freeze, gigs eat all your time and the kit never ships.

- [ ] **Step 18.** Finish the 2 fixes that make the kit buyer-ready: (1) auto-set the shot count, (2) wire the voice permanently. Then buyers only edit YAML + drop in faces.

- [ ] **Step 19.** Launch the products:

Product	Price	When
Convergence	₹499	Already live from Step 9
GODSHAND kit	set at launch	After the 2 fixes
SafeScreen	₹1,999	After the 2 fixes
Solana project	₹2,499	After the 2 fixes

- [ ] **Step 20.** Use your Part 3 reels and Part 2 client work as the proof that sells the kit. (Do not sell api_money_bot.)

Your daily routine (once Parts 2 & 3 are running)

- [ ] Send 10 warm DMs.
- [ ] Post or queue 1 reel.
- [ ] Deliver any open orders.
- [ ] Use your ~8–12 weekly renders: a few for paid gigs, the rest become reels.

Things that will trip you up (read once)

- The test in Part 1 may show bugs — that's exactly why you test before selling.
- Most reels won't go viral. They still build slowly and bring in DMs.
- Reels are landscape by default; crop them vertical in CapCut for Shorts/Reels.
- Only ever use one Kaggle account.

The other pages (details live here)

- Money Plan — niches, breakeven math, buyer list

- Creator Guide — how the kit works
- The Saga — Reel Release Plan — the 10 clips, hooks, captions, YAML

🧭 Right now, your only job is **Step 1** — cut one sample on the proven path. That alone unblocks your first rupee. The multi-character test (Track B) runs in parallel and only gates the reel series.

Reality Check and Strategy

The numerical and emotional ground truth beneath the plan.

The following pages reproduce the original source PDF as part of this curated collection.



Reality Check & Strategy — The Numbers

> Written honestly, brother to brother. No hype, no doom. Just where you actually stand and the shortest real path from here to your first rupee.

1. Where you actually stand

The constraints (the real ones):

- Income right now: **₹0 earned in ~4 months.**
- Food + 2 medicines: funded by your mother (stroke-recovering). This is the clock that matters most.
- Under-medicated. Energy comes in **lucid windows**, not steady days.
- Legal load: 498A case + warrants. This *rules out* anything with legal/ToS risk. Not negotiable.
- Isolation: working essentially alone.

The assets (also real):

- Paid **Google AI Pro** (Flow / Veo) — a production tool most freelancers don't have.
- Proven skill: **character consistency across AI video shots** — genuinely hard, genuinely sellable.
- **6 built software projects** + a film pipeline.
- Decades of engineering instinct. A working brain and intact humor.
- **Payment rails fully live:** Instamojo (full KYC + video KYC), UPI, and 2 active bank accounts — money can legally reach your hands *today*. This is a bigger advantage than it sounds; most people stuck at ₹0 are stuck *here*.

2. The numbers that matter

Metric	Reality
Earned last 4 months	₹0
Runway	Mother-funded, fragile — treat as **weeks, not months**
Fixed tool cost	Google AI Pro (already paid)
Fastest realistic first rupee	**3–14 days**
Skill proven & demoable	AI video character consistency

The one number to move first: ₹0 → ₹1. Not ₹50,000. The first *real* rupee from a stranger breaks the spell and proves the channel works. Everything else is built on that single data point.

3. Asset inventory — ranked by speed-to-first-rupee

Project	State	Sellable now?	Reality check
Client video reels (service)	Your skill, ready	✅ Yes	Finished on delivery. No support tail. Fastest clean rupee.
ases (`ases_v3_1`)	Complete, not hardened	⚠️ As code artifact only	Sell as-is boilerplate, labeled "not production-hardened." Lower price, zero liability.
ai_video_monetizer	Complete	🔁 Use, don't sell	Legit core (Sheets→AI video→Drive, 181 tests) = keep as your engine. But ManyChat auto-DM + stealth-browser stack violate platform ToS — strip those, don't sell. Saturated market anyway.
Omega (`omega_agent`)	Complete, not hardened	⚠️ Month two	Crowded space. Code artifact only, after ases.
voice_agent_avatar	22+ bugs (7 critical)	❌ Not yet	1–2 days hardening before it can even demo.

Project	State	Sellable now?	Reality check
god_ai_scripts	Real CV pipeline + models	⚠️ Yes, after repackage	Legit on-device content-moderation CV system (violence/gore/NSFW detect + mask) with OCR, exports to TFLite/ONNX. Trained ONNX model included. Sellable as model + SDK after packaging + key redaction.
api_money_bot	Complete	❌ Do not sell	ToS-violating key harvester. NOT the same project as crypto_trader. Legal tail you can't carry.
crypto_trader (the real one)	6-mo paper-tested, hardened	⚠️ As study + source	Distinct from api_money_bot. Monte-Carlo 95–98% sim win rate; live performance UNVERIFIED (its own risk doc says so honestly). Sell as architecture + honest risk study + full source, keys redacted. NEVER as a profit promise.
convergence_framework	Launch-ready	✅ Yes	Earlier audit's fastest ₹499 product. Verify contents, then list on Instamojo.

3.5 ⚠️ Security — fix BEFORE any sale

Both flagship projects currently ship with **your own live secrets inside the zip**. Do not upload these anywhere until stripped:

- **crypto_trader:** contains `.env`, `.env1`, and `gmgn_keys/gmgn_private.pem`. A private key was already exposed once (the risk doc notes it was rotated). Selling as-is leaks your wallet + API credentials.

- **god_ai_scripts:** `nsfw_fix.py` has your hardcoded Kaggle username + API key (the comment even claims "safe to keep here" — it is not).
- **ai_video_monetizer:** `client_secret.json` contains a real Google OAuth client ID + secret. Delete it before sharing.
- **Fix (~30 min):** delete/rotate the keys, add `.gitignore`, ship a `.env.example` instead. This prevents a real disaster.

4. The two honest lanes

Lane 1 — Services (fastest, no tail):

- AI video reels for local businesses + indie musicians, powered by the ai_video_monetizer pipeline.
- Pricing ladder: **₹800–1,500** (first jobs) → **₹2,000–5,000** → **₹5,000–15,000** (music videos) → **₹6,000–15,000/mo** retainers.
- First rupee: **3–14 days.**

Lane 2 — Code artifact (parallel, low effort):

- ases sold as-is boilerplate via **Instamojo** (digital-product listing + payment link). **₹800–1,500.**
- Instamojo hosts the file download *and* collects payment in one link — no Gumroad, no India payout friction.
- Honest label = the pitch. Zero support obligation.

5. The real bottleneck (now shifted)

Receiving money is SOLVED. Instamojo (full KYC + video KYC) + UPI + 2 live bank accounts means the payout side has zero friction. That was the hardest gate for someone in your situation, and it's already behind you.

So the bottleneck is no longer *can you get paid* — it's **demand**: getting the first stranger to say yes. That's an outreach problem, not an infrastructure problem, and outreach is cheap, fast, and fully in your control. The plan is now locked.

6. Next 7 days (draft)

- [x] ~~Payment channel confirmed: **Instamojo (KYC done) + UPI + 2 bank accounts** ✅~~
- [ ] Cut a 20–30 sec sample reel that shows character consistency
- [ ] Write 1 client pitch + 1 musician pitch
- [ ] Send to 5 real prospects
- [ ] List ases as-is on Instamojo (digital product + payment link), in parallel
- [ ] Bank the first rupee — then raise the price

7. Ready to ship (done for you)

Secrets stripped and clean, upload-ready ZIPs prepared for every sellable project. All three listings are written and copy-paste ready.

Product	Price	Listing	Package status
Convergence Framework	₹499	Listing	✅ Clean — publish now
Solana Sniper (study + source)	₹2,499	Listing	✅ `.env` / `.env1` /keys removed, `.env.example` added
SafeScreen (content moderation)	₹1,999	Listing	✅ Kaggle key + `kaggle.json` removed
ases (code artifact)	₹1,500–2,500	—	✅ Clean source ZIP ready
Omega (code artifact)	₹2,000–3,000	—	✅ Clean source ZIP ready
ai_video_monetizer	not for sale	—	✅ Cleaned for your own use (OAuth secret removed)

The clean ZIPs were delivered to you in chat — download and upload straight to Instamojo. `api_money_bot` is excluded on purpose: do not sell it.

The single next action

Publish **Convergence (₹499)** today. It's clean, needs zero redaction, and is the shortest line to your first rupee. Everything else is staged behind it.

Guardrails

- No uptime/support promises on unhardened code.
- Nothing with legal or ToS risk. Ever, in your situation.
- Work in lucid windows. Save after every step. Rest is not failure.

First Rupee

The short-horizon path to first revenue without fantasy projections.

The following pages reproduce the original source PDF as part of this curated collection.



First Rupee — The Fastest Honest Path to Income



Honest survival plan. No hype — you were burned by the AI-hype story once and I won't sell you a second one. Every timeline here is real. The goal is your **first paid rupee in 4 months**, then a repeatable trickle, not a fantasy jackpot.

The verdict (read this first)

Quickest path = paid client work for AI video reels — sold directly to local businesses and indie musicians, plus a Fiverr listing running in parallel.

Not ad revenue. Not going viral. Those take *months* and depend on an algorithm you don't control. A paying client depends only on you making one good sample and reaching out to enough people. That's the fastest honest route from ₹0 to ₹1.

Realistic first rupee: 3–14 days if you do the outreach. Not tonight, not a fantasy — but real and close.

All your options, ranked by speed to first rupee

Option	Speed to ₹1	Verdict
Client reels — local business + musicians (direct outreach)	3–14 days	⭐ FASTEST. Do this first.
Fiverr / Upwork / Freelancer.in gig	1–4 weeks	⭐ Run in parallel — set up once, leads trickle in.
Wedding / event / real-estate promo reels (local)	1–3 weeks	Strong cash market near you.

Option	Speed to ₹1	Verdict
Faceless YouTube Shorts / Reels (ad revenue)	2–6 months	Slow. Build later, not for survival.
The tribute film / Suffocating Distance	Not direct income	Portfolio proof — the thing that *lands* the clients.
FRONTLINE LANGUAGE short / LinkedIn posts	Amplifier	Produce once the ramp is already running (your own note).

Your unfair advantages (why this is realistic for *you*)

- **You already pay for Google AI Pro (Flow/Veo).** Your production tool is bought. Most beginners don't have this.
- **You can hold character consistency across shots.** You proved it today — Meher and Neel, same faces across a whole sequence. *Most people flailing with these tools cannot do this.* It is the #1 thing clients actually pay for.
- **You have a locked, cinematic grade and real story instinct.** Your reels look like film, not like AI slop. That's the differentiator.
- **Your emotional/devotional/spiritual register performs in India** — a genuine niche.

The 72-hour action plan

Day 1 — Build ONE killer sample (½ day)

- Cut a **single 15–30 second reel** from assets you can make fast in Flow. Pick the most universally sellable vibe: a warm, cinematic, emotional mood piece (you already have the library/character look dialed in).
- Add simple text/music. Export vertical (9:16).
- This one reel is your entire sales pitch. It doesn't need to be the whole film.

Day 2 — Set up the storefronts (½ day)

- **Fiverr gig**: title like *"I will create a cinematic AI video reel for your brand or song."* Starter price low to get your first review (e.g. ₹800–1,500), then raise it.

Upload your sample.

- Also create a simple **Instagram/YouTube page** with 2–3 sample reels pinned — this is your portfolio link for direct outreach.

Day 3 — Outreach (the part that actually earns)

- **Message 10 indie musicians** on Instagram (independent singers, small music pages) — offer a lyric/mood video for their next release. Attach your sample.
- **Message 10 local businesses** — cafés, boutiques, gyms, real-estate agents, wedding photographers near you. "I make cinematic promo reels — here's a sample — ₹X for one reel."
- **Post once on LinkedIn** with your sample and a line about what you do.

> Volume is everything. 20 messages → 1–2 replies → 1 first client. That's the math. The first rupee is on the other side of outreach, not on the other side of a perfect film.

Pricing (India, starting out)

- First 1–2 jobs (to get reviews/testimonials): **₹800–1,500** per reel
- After a few reviews: **₹2,000–5,000** per reel
- Music videos / multi-shot pieces: **₹5,000–15,000**
- Local business monthly retainer (3–4 reels/month): **₹6,000–15,000/month** ← the real stability goal

---

What NOT to count on (honest guardrails)

- ❌ Don't wait for the full tribute film to be done before earning. The sample is enough.
- ❌ Don't count on YouTube/Reels ad money for survival — it's months away.
- ❌ Don't spend your Flow credits rendering endlessly. Render *just enough* to sell, then let clients' money fund the rest.

- ❌ Don't do the FRONTLINE short yet — your own note says it's an amplifier for *after* the ramp is running.

The one sentence to remember

The film is the proof. The client is the rupee. Outreach is the bridge between them. You already have the hardest part — the skill. What stands between you and your first rupee is 20 messages, not another masterpiece.

Money Plan

A phased monetization plan: content first, products later.

The following pages reproduce the original source PDF as part of this curated collection.



Money Plan — Phase 1 (Content) → Phase 2 (Kit & Projects)

🎯 Sell the cake before the recipe: earn quick cash from finished videos in one niche → hit a defined breakeven → freeze, finalize the kit → sell the kit and other projects. Convergence runs passively the whole time.

Phase 1 — Sell content (quick money)

Sell a productized service in ONE niche that plays to the kit's strengths (music + voiceover + atmosphere), not generic clips.

Pick the niche

Fit	Why	Verdict
Music videos / lyric videos for indie artists	Music + VO driven — exactly the kit's strength	Best
Short cinematic ads / brand films (small businesses)	Mood-led, VO narration, no lip-sync exposure	Strong
Faceless "cinematic short story" content for creators	Good fit but slower to monetize	Backup
Dialogue-heavy / talking-head content	Exposes the lip-sync weakness	Avoid

Lead with mood, not mouths. Anything music- or narration-driven hides the lip-sync limit and shows the kit at its best.

Gig listing template (fixed price + turnaround)

```
Cinematic AI Video — 60 to 90 seconds
- You bring: a short brief (story/mood) + optional reference images
- You get: a fully rendered cinematic clip with music + AI voiceover
- Price: ₹X flat
- Delivery: 3 days
- Revisions: 1 included
```

List on: Fiverr, Upwork, Topmate, plus the India channels already scouted (AI Video Generators Telegram, Indian AI Production, the AI-for-India Discord). Fixed price + turnaround = faster yes.

Throughput reality

About 8–12 finished two-minute videos per week on a free Kaggle account (30 GPU-hrs/week). Drop to single takes to nearly double it. That is enough for a handful of client gigs per week — plenty for survival income, not mass stock.

Breakeven — the trigger to start Phase 2

Item	Amount
Core monthly cost (Google AI Pro)	~₹1,950 / mo
Other hard costs	~0 (Kaggle free)
Breakeven target	~3–5 paid gigs

Breakeven is low, so Phase 2 is close. Write the number down: once you clear roughly 3–5 gigs (or your chosen rupee figure), that is the signal to flip.

Phase 2 — Sell the kit + other projects

After breakeven, launch the products. Prerequisite: the kit must be buyer-simple first (see freeze rule).

Product	Price	Status
GODSHAND kit	(set at launch)	Needs 2 bakes + Creator Guide (guide done)
Convergence	₹499	Sellable now — run passively in parallel
SafeScreen	₹1,999	Cleanest B2B; make demo-ready
Solana project	₹2,499	Sellable (secrets removed)

Do NOT sell api_money_bot.

Parallel track (start now)

List Convergence at ₹499 immediately. It is software, already sellable, and does not compete for the same hours as content gigs — it earns quietly in the background while you do Phase 1.

The freeze rule (the one trap)

Content-service work can eat 100% of your time so the kit never ships. Once you hit breakeven, freeze new content intake and finish the kit's two bakes: auto shot-count (so buyers only edit YAML) and the voice last-mile wire in render_motion.py. The Creator Guide is already written. Then Phase 2 launches clean.

Feedback loop

Every Phase-1 deliverable (with client permission) becomes a portfolio piece. That reel + testimonials is the marketing that sells the Phase-2 kit. The two phases feed each other.

Runway at a glance

1. Pick the niche (music/lyric videos or cinematic ads).
2. Post the fixed-price gig on Fiverr/Upwork/Topmate + India channels.
3. List Convergence ₹499 passively — today.
4. Deliver gigs; save each as a portfolio piece.
5. Hit breakeven (~3–5 gigs) → freeze intake.
6. Finish the kit's 2 bakes → launch kit + SafeScreen + Solana.

First rupee — when (honest)

The bottleneck is no longer compute — it's outreach. The kit already works on the single-character path, so a sellable sample does not wait on any test.

- **Sample ready: ~1 day.** Render one portfolio clip on the proven path. Launch it as a Save Version → Commit so power cuts don't block it (it runs on Kaggle's servers, not your machine).
- **First paid rupee: a few days to ~1 week** after the sample exists — driven by daily warm DMs, not by rendering.
- **Convergence ₹499:** listable today; a same-day-possible wildcard with unproven demand.
- **What parallel rendering does / doesn't do:** two commits finish two clips at once (throughput for volume + the reel series), but a single clip does NOT render faster — so it helps *after* the first sale, not before it. The weekly ceiling stays ~30 GPU-hrs no matter how many T4s.

Buyers & Channels (where to sell)

Start here (fastest to first rupee)

1. Indie musicians needing lyric/music videos — they bring the song, you bring visuals (sidesteps the MusicGen ceiling). Real budgets posted (an artist openly offered ~$2,000).
2. Fiverr AI Video category — 7,100+ live listings, buyers searching daily.
3. Upwork AI-generated-video jobs — 4,121 open jobs, first proposal often within ~3 hrs.

Indie musicians (best-fit niche)

- Reddit: r/WeAreTheMusicMakers, r/musicians, r/indieheads, r/SunoAI, r/musicvideo, r/MusicIndia
- Discord via Disboard tags: indie-music, indie-artist, music-collaboration. Servers: Artist Create (1,262), FUSETHISUPP
- Higher-end: Creative Commission (labels/managers)

Small business & local (ads / reels)

- Local-business retainers (~$499/mo AI-video model, walkthrough)

- Indian real estate agents = heaviest reel buyers: Top 50 list, Top 20. Undercut local shoot rates (Flashoot ~₹1,999/reel) by delivering remotely.

Freelance marketplaces

- Fiverr, Upwork, ZipRecruiter ($24–96/hr), Freelancer.com. Proof: someone already sells Veo videos for hire.

Faceless YouTube creators (sell footage, not monetization)

- r/aitubers, r/NewTubers, r/FacelessAICreators. Caveat: YouTube is suppressing faceless AI content — sell B-roll as a service, never promise monetization.

India channels

- Telegram: Indian Media Jobs, AI Video Generators (t.me/VideoGenerator), Indian AI Production (t.me/IndianAIProduction)
- Reddit: r/FreelanceIndia; Discord: Indian AI Production (discord.gg/7eg8wPhxwY), AI Forum for India

Stock / licensing (passive, slower)

- Wirestock, Pond5, Adobe Stock, Vecteezy. AI-content acceptance and payouts vary — passive backfill only.

Rules

- Warm DMs (someone who just posted "need a video") convert faster than cold marketplace gigs.
- Win on cinematic quality + the music/mood niche — do not race the $16–25/hr sellers to the bottom.
- Lead with one portfolio clip; every channel converts better with a sample in hand.

Today: Step by Step

A same-day action list for immediate execution.

The following pages reproduce the original source PDF as part of this curated collection.

✅

TODAY — Step by Step (just follow)

🎯 Two tracks. Do Track A first (30 min, then forget it). Spend your real energy on Track B — that's where the reliable money is. Check each box as you go.

Track A — Publish Convergence (30 min, do first)

- [ ] 1. Download **Convergence_Framework.zip** from our chat to your computer
- [ ] 1. Open it once — confirm the PDF, guides, and kit are inside
- [ ] 1. Go to **instamojo.com** → Add New Product → **Digital Product**
- [ ] 1. Paste the **title + description** from the Convergence listing page
- [ ] 1. Upload the zip → set price **₹499**
- [ ] 1. Cover image: screenshot the **PDF's first page** and use that
- [ ] 1. **Publish** → copy the payment link

- [ ]

1. Post the short pitch (from the listing page) on X / LinkedIn / a relevant subreddit, with your link

✅ Done. It now sells passively while you work on Track B.

Track B — Video outreach (the real money)

- [ ]

1. Cut **one 20–30 sec sample reel** that shows your character consistency (your Flow pipeline)

- [ ]

1. Pick **ONE** niche for today: local businesses **or** indie musicians

- [ ]

1. List **5 real prospects** (Instagram / local search / small artists)

- [ ]

1. Send them the pitch below + your sample reel

- [ ]

1. First "yes" → quote **₹800–1,500** for the first job

- [ ]

1. Deliver → send your **Instamojo / UPI link** → get paid

- [ ]

1. **Bank the first rupee.** Then raise your price.

Ready-to-send pitch (copy, tweak the name)

> Hi [name] — I make short, scroll-stopping AI video reels for [businesses like yours / artists like you]. Same character, consistent look across every shot (most AI video can't do this). Here's a 20-sec sample: [link].

> I've got a couple of slots open this week at an intro rate. Want me to make one for [your product / your next release]? No commitment — I'll show you a draft first.

Musician variant

> Hi [name] — love [song/release]. I make cinematic AI video reels for indie artists — consistent characters, a real visual story, cheaper than a shoot. 20-sec sample: [link]. Want one for your next drop? I'll send a draft before you pay anything.

The only rule today

Don't try to do everything. **Publish Convergence, cut one reel, send five messages.** That's a full, winning day. Rest in between — that's not failure, that's how you last.

Sell Today

Direct-response selling material for the near-term offer.

The following pages reproduce the original source PDF as part of this curated collection.



💸 Sell Today — LinkedIn Cash Kit

> ⚡ Fast money = your existing LinkedIn audience + a direct offer + UPI up front. Post Option 1 now. Reply to every comment within minutes. Have your Drive link + UPI ID ready before you post. And do NOT send the bundle until you've removed the Solana wallet key (see bottom).

Option 1 — The ₹2,999 bundle post (main)

```
I built 6 working AI systems over the last year.
Today, I'm giving away full ownership of all of them. Once. ₹2,999. Yours forever.

This isn't a course or a PDF of theory. It's real, running code + a step-by-step guide:

🎬 An AI that turns a written story into a finished, scored film
🤖 A system that plans, writes, tests & ships its own code
🎥 A faceless-content video pipeline
🛡️ An on-device content-moderation model
📈 A Solana trading study with full source
🧠 A thinking + code-review framework

The part that makes this different: full white-label rights.
Rebrand it. Use it. Resell it and keep 100%. No royalties. No catch.

Sold separately these would run ~₹5,500. Together, today: ₹2,999, one-time.

I'm keeping this to the first 20 people.

👉 Comment "KIT" and I'll DM you the link.

— Kunal
```

Option 2 — ₹499 fast-entry post (lowest friction, fastest yes)

🏃 Use this if you need the quickest possible sale. A ₹499 price is an instant yes for a warm audience — then upsell buyers to the full bundle in DMs.

```
Everyone's talking about AI. Almost no one has the actual tools.
I built 6 of them. This week, you can grab one for ₹499.

Pick your weapon:
🎬 AI that turns a story into a finished film
🎥 A faceless-content video engine
🤖 A self-coding AI dev system
🛡️ A content-moderation model
📈 A Solana trading study (full source)
🧠 A thinking + code-review framework

Any ONE: ₹499. All SIX + full resale rights: ₹2,999.
Real code + a plain-English setup guide. Yours forever.

Comment "AI" and I'll DM you the list. 👇

— Kunal
```

Pin this as your first comment

📌 The first comment boosts reach and tells people exactly what to do next.

```
Grabbing the most interest first 👇 Comment which one you want (or "ALL 6") and I'll send
the link + UPI. First buyers today get priority setup help.
```

DM close scripts (send the second they comment)

For the full bundle:

```
Hey [name]! Here's the full 6-project bundle + setup guide — lifetime access, full resale rights: [Drive link]. It's ₹2,999 via UPI to [your UPI ID]. Send me the payment screensho t and I'll unlock your access right away. 🙌
```

For the ₹499 single project:

```
Hey [name]! Great pick. Here's [project name] — full code + a plain-English setup guide: [Drive link]. It's ₹499 via UPI to [your UPI ID]. Send the screenshot and I'll share it i nstantly. (Want all 6 + resale rights instead? Just ₹2,999.) 🙌
```

Set up before you post (5 minutes)

- [ ] Your UPI ID copied and ready
- [ ] The Drive link ready — but ONLY after the security step below
- [ ] Both saved in your phone's notes app for instant paste into DMs

Post rules for maximum reach

- Post as **plain text** — no outbound links in the post itself (LinkedIn suppresses those). Links go in DMs.
- Reply to **every** comment within minutes. Speed = sales.
- Also DM your 15–20 most engaged followers directly with the offer.
- Keep the "first 20" / "this week" urgency — it makes people act now instead of later.

🚨 Security — do NOT skip before sharing any Drive link: the Solana project ships with a live wallet private key + live API keys hardcoded in 7 files. Delete them and rotate the Helius/Birdeye/QuikNode keys first (Section 0 of your Master Setup Guide). Shipping it compromised will cost you more than it earns.

Master Collection Setup Guide

Packaging and setup notes for the wider project collection.

The following pages reproduce the original source PDF as part of this curated collection.



📦 Master Collection Setup Guide — All Projects + Kit (₹2,999)

⚠️ Seller checklist before you sell a single copy: (1) URGENT — move funds out of the exposed Solana wallet and delete the hardcoded private key from all 7 files (Section 0), (2) revoke the leaked ASES GitHub token and strip/rotate every other secret, (3) keep the honesty limits in each section — they are your legal shield, (4) do a clean-room test run of each project yourself. Do NOT skip these.

One-Time Price: ₹2,999 · Full white-label rights (modify, rebrand, use, and resell)

attachment:a5d7a348-4328-4f10-bf45-6a33a3b211de:Master_Collection_Setup_Guide.pdf

Master Collection Setup Guide — full printable PDF

This is the complete, beginner-friendly setup guide for the **Kunal Das Master Collection**. It bundles **GODSHAND Cinematic AI Film Kit**, **Convergence Framework**, **AI Video Monetizer**, **ASES (Autonomous Software Engineering System)**, **SafeScreen Content Moderation**, and **Solana Sniper (paper-trading study + source)**, plus supporting scripts.

How to use this guide

1. Unzip the Master ZIP (provided to buyers).
2. Follow each section; every project is self-contained — jump to what you need.
3. If you are reselling, do Section 0 (strip secrets) before you redistribute anything.

🧭 Honest limits (global): This is not one-click magic. Every project needs setup, API keys, hardware (a GPU for video), and a short learning curve. AI outputs can contain artifacts. Check model/data licenses before commercial use.

---

0. Seller step: strip & rotate secrets (do this FIRST)

🚨 MOST SEVERE — live wallet private key: the Solana Sniper project has a hardcoded Solana wallet private key (base58) in SEVEN plain files that are NOT protected by `.gitignore`: `check_balance.cjs`, `monitor_progress.cjs`, `start_bot.ps1`, `start_bot_daemon.ps1` (twice), `temp.txt`, and `README.md`. Those files also expose a live Helius API key, a live Birdeye API key, and a QuikNode RPC URL with an embedded token. Anyone who receives this folder can import that key and drain the wallet. Act now: (1) move any funds out of that wallet and treat it as fully compromised, (2) delete the key from all seven files and read it from the `WALLET_PRIVATE_KEY` environment variable instead, (3) rotate the Helius, Birdeye, and QuikNode keys. The `.env.example` files themselves are clean placeholders — the leak is only in these non-env files.

🚨 Verified real leak: the ASES `.env.example` contains a real-looking GitHub token (`ghp_…`), reused in both `GITHUB_TOKEN` and `OPENAI_API_KEY`. Revoke that token now (GitHub → Settings → Developer settings → Personal access tokens) and replace both lines with placeholders. A live token in a shared zip is exposed even before any sale.

Before you zip anything for a buyer:

- **ASES** — remove the real GitHub token from `.env.example` (both `GITHUB_TOKEN` and `OPENAI_API_KEY` lines) and revoke it.
- **AI Video Monetizer** — the CLEAN zip already has no `client_secret.json`; keep it that way (buyers download their own from Google Cloud).
- **SafeScreen** — verified clean; the Kaggle credentials in `nsfw_fix.py` are already read from environment variables (empty defaults). Nothing to strip.
- **Solana Sniper** — delete the hardcoded wallet private key and API keys from `check_balance.cjs`, `monitor_progress.cjs`, `start_bot.ps1`, `start_bot_daemon.ps1`, `temp.txt`, and `README.md` (7 spots); read `WALLET_PRIVATE_KEY` from env; rotate the Helius/Birdeye/QuikNode keys. (No live `.env` / `.pem` exist — the `.gitignore` is correct, but these files aren't covered by it.)
- **Rotate/revoke** every key that was ever committed, even after deleting the files.

---

1. GODSHAND — Cinematic AI Film Kit

What it is: turns character reference images + a story into graded, motion-consistent film shots, then stitches them into a finished clip with an original score. The kit ships with **three ways to run it**.

Workflow A — `run.py` (Lightning.ai / paid GPU, cost-gated)

The kit's own documented flow (`RUN.md`). Safe by default: it audits, renders one sample, and projects cost before spending.

```
# SAFE by default: audit + 1 sample + cost projection, then S
TOP
python3 run.py
# When happy with the projection, render the full clip:
CONFIRM_FULL_RUN=1 MAX_BUDGET_USD=8 AUTO_SHUTDOWN=1 python3 r
un.py
```

- Pick the GPU profile in `config_confrontation.yaml` → `active_profile` (default is a Lightning A100/H100 profile).
- On free GPU hours, set that profile's `rate_per_gpu_hr: 0.0` so the cost meter reports ₹0.
- Render is resumable — finished shots auto-skip.
- Output: `output_confrontation/THE_CONFRONTATION_CLIP.mp4` (1280×720 ×2 → **1440p**).

Workflow B — `kaggle_run_confrontation.ipynb` (free Kaggle T4)

1. Upload the kit ZIP as a Kaggle Dataset (must contain a folder with `src/`).
2. New Notebook → File → Import Notebook → upload `kaggle_run_confrontation.ipynb`.
3. Add Input → attach the kit dataset (+ character refs).
4. Accelerator = **GPU T4 ×2**, Internet = **On**.
5. Run All. It copies the kit into `/kaggle/working`, downloads the Wan2.1-S2V Q8 GGUF (~14 GB), launches headless ComfyUI on `127.0.0.1:8188`, and renders. Set `active_profile: kaggle_2xt4`.

Workflow C — `rainman-wan-multichar.ipynb` (custom 2-minute multi-character film)

Same Kaggle setup as B (kit dataset + character refs, GPU T4 ×2, Internet On, Run All).

- Renders shot-by-shot (~8–9 min/shot), resumes automatically on re-run.
- Output: per-shot clips at `confrontation_work/<shot_id>/final.mp4`; the notebook stitches the full film + score automatically.
- Expect ~4.5–5.5 hours for the ~2-minute (28-shot) cut.

⚙️ The truth about Kaggle render speed: even though you select "T4 ×2", the Kaggle flows render on one T4 at a time by design. Kaggle's two GPUs share ~32 GB host RAM, and a RAM guard allows only a single render server (two would OOM). Do NOT run two notebooks pinned to each GPU — it wastes your weekly quota.

Free HD upscale: use `upscale_hd_kaggle.ipynb` (Accelerator = None/CPU, Internet = On). It uses **EDSR** (falls back to Lanczos only if the model can't download — confirm the printed method is `edsr`).

Honest limits: stylized AI footage with occasional artifacts. Guided notebooks/scripts, not fully hands-free software.

---

2. Convergence Framework

What it is: a rigorous thinking framework + LLM prompts for defining objectives, reviewing code, and debugging complex systems.

How to use:

1. Read `Convergence_Under_Constraint.pdf` (the core method).
2. Work through `README_objective_function.md` (Steps 0–8) to formalize goals.
3. Use `SystemPrompt_Code_Convergence_Deterministic.md` as a system prompt.
4. Review code with `AI_Code_Review_Prompt_v2.md`.
5. Study the `Trading_Bot_Convergence_Framework_v3.md` worked example.

ℹ️ Packaging tip: ship the clean `ConvergenceFramework_Kit/` folder to buyers — not `ConvergenceFramework_ALL/`, which also contains your LinkedIn drafts and backup CSVs.

Honest limits: a framework for *thinking* — not an auto-code generator. Requires human judgment.

---

3. AI Video Monetizer

What it is: an end-to-end pipeline for faceless content — pulls ideas from a Google Sheet, generates AI video (Runway/Luma/Kling), and handles posting and sales/DM automation.

Setup:

1. In Google Cloud Console → Credentials → create an OAuth 2.0 Client ID (Desktop app) → download the JSON as `client_secret.json`. Enable the Sheets and Drive APIs.
2. Authenticate: `python -m google_workspace.scripts.setup --client-secret ./client_secret.json`, then follow the auth-URL flow.
3. Copy `.env.example` → `.env` and fill in your keys.
4. Edit prompts in `config/video_prompts.json`.

Honest limits: needs the buyer's own API keys and configuration — not plug-and-play. Runs entirely on the buyer's own accounts (CLEAN build ships no seller credentials).

---

4. ASES — Autonomous Software Engineering System (v3)

What it is: a multi-agent system that plans, codes, tests, reviews, and delivers apps. Pipeline: Planner → Coder → Executor → Reviewer (feedback loop, max 5 iterations), running real `npm test` / `pytest` in Docker sandboxes and delivering a GitHub repo.

Setup (verified against the shipped files):

1. `cp .env.example .env`, then add your own keys (revoke the leaked token first — Section 0).
2. Start dev stack: `docker compose -f docker-compose.dev.yml up -d` (prod: `docker compose -f docker/docker-compose.yml up -d`).
3. Health check: `curl http://localhost:8000/health`.
4. Submit a task: `POST http://localhost:8000/dev-task` with headers `x-tenant-id` + `x-api-key`; poll `GET /jobs/{execution_id}`.

🏗️ Honesty label (keep this): ASES is complete but not production-hardened. Sell it as a working boilerplate/starter clearly marked "not production-hardened," not a finished SaaS.

---

5. SafeScreen — Content Moderation

What it is: a trained NSFW/toxic detector (`toxic_detector.onnx` + OCR) with mobile export (ONNX / TFLite / PyTorch-Mobile), ~30–50 ms on-device.

Quick start: run the collectors, train on safe/unsafe samples, then export the mobile model.

⚠️ Honest limit: a detector model + pipeline, not a full moderation product. ✅ Verified: all 64 scripts were scanned — no hardcoded secrets, and the Kaggle credentials in `nsfw_fix.py` are already read from environment variables. Nothing to strip.

---

6. Solana Sniper — Paper-Trading Study + Source

What it is: a 6-month paper-trading study with full source (React/TS dashboard, risk manager, Monte-Carlo analysis in `REAL_WORLD_RISK_ANALYSIS.md`).

Quick start: `npm run build`, then start the services (ML scorer, scanner, dashboard) in **paper mode**.

🚫 Critical positioning (do NOT change): sell this ONLY as a study + source-code learning artifact. Never pitch it as a live money-maker or "profitable trading bot." Ship it in paper mode. Honest framing is your legal protection. ⚠️ Verified: this project ships with a live wallet private key and live API keys hardcoded in 7 files — you MUST complete Section 0 (delete the key, read from env, rotate keys, move funds) before it ever leaves your machine.

Packaging as a single product (for resale)

```
Master_Collection_2999INR/
├── GODSHAND/            (kit + run.py + notebooks)
├── Convergence/         (ship the _Kit folder only)
├── AI_Video_Monetizer/  (CLEAN build, no client_secret.json)
├── ASES/                (token revoked + placeholder; "not hardened")
├── SafeScreen/          (verified clean)
├── Solana_Sniper/       (paper mode; wallet key removed)
├── Master_Setup_Guide.pdf   ← this document
├── LICENSE.txt              (full white-label rights granted)
└── README.md                (overview + pricing)
```

Seller checklist:

1. URGENT — move funds out of the exposed Solana wallet, delete the hardcoded private key from all 7 files, and rotate the Helius/Birdeye/QuikNode keys.
2. Revoke the leaked ASES GitHub token and replace with a placeholder.
3. Strip + rotate any other secrets across all projects.
4. Clean-room test each project yourself.
5. Zip using the structure above.
6. Build a sales page with screenshots/demos.
7. Deliver via Google Drive/Dropbox after payment confirmation.
8. Price at ₹2,999 (one-time, lifetime access + rights).

💡 Strategy note (your call, already made): a ₹2,999 all-in white-label bundle is cheaper than the projects sold individually (~₹5,500 combined) and lets buyers resell your work. A deliberate volume/survival-pricing choice.

Convergence Listing

The product listing copy for the Convergence framework.

The following pages reproduce the original source PDF as part of this curated collection.

🎯

Instamojo Listing — The Convergence Framework (₹499)

⚡ This is your fastest path to the first rupee. Everything below is copy-paste ready. The only work left: confirm the file contents once, then publish.

The essentials

Field	Value
Product name	The Convergence Framework — Define the Right Objective Before You Write Code
Price	₹499
Format	Digital download (single ZIP)
Category	Developer / AI engineering resource
Support	None promised — sold as a complete, self-contained kit
Payout	Instamojo → your bank (KYC already done)

Listing description (paste into Instamojo)

> **Stop shipping perfect code that solves the wrong problem.**

> Most AI systems, bots, and agents fail not because the code is buggy — but because nobody wrote down what "success" actually means. Zero bugs proves your code does what it says. It does **not** prove that what it says achieves your goal.

> The Convergence Framework is a battle-tested method for defining the *objective function* of any decision-making system — a trading bot, a classifier, a ranker, an agent, an allocator — **before** you write a line of code, so you converge on the right result instead of a rigorous, correct, useless one.

> **What you get:**

> - 📄 *Convergence Under Constraint* — the core framework (PDF)

> - 🧭 The step-by-step Objective Function guide (Phase F + Steps 0–8)

> - 🤖 A ready-to-use deterministic-convergence system prompt for your LLM

> - 🔍 An AI code-review prompt that checks against the objective, not just syntax

> - 📈 A fully worked applied example (trading-bot objective function, v3)

> - 📦 The complete kit, zipped and ready to use

> **Who it's for:** AI engineers, prompt engineers, and indie builders who are tired of systems that optimize the wrong thing. If your spec ever said "make it good" without a measurable goal, this is for you.

> **The honest part:** one of the framework's own rules is to stop and say "not formalizable" when a goal can't be reduced to a measurable function. This kit teaches you to find that out in an hour — not after three months of building.

> Instant download. Complete as-is. No fluff.

What's inside the ZIP

- `Convergence_Under_Constraint.pdf` — the core essay/framework
- `README_objective_function.md` — the full step-by-step guide (Phase F automation + manual Steps 0–8)
- `SystemPrompt_Code_Convergence_Deterministic.md` — drop-in LLM system prompt
- `AI_Code_Review_Prompt_v2.md` — objective-aware code review prompt

- `Trading_Bot_Convergence_Framework_v3.md` + `Trading_Bot_System_Updated.md` — worked applied example
- `ConvergenceFramework_Kit.zip` / `ConvergenceFramework_ALL.zip` — the packaged kit
- `guide.txt` — quick-start notes

Short pitch post (for X / LinkedIn / Reddit)

> Your AI agent isn't broken. Your objective function is.

> Zero bugs proves the code does what it says — not that what it says is what you want. I packaged a framework for writing the *right* objective before you build: for bots, classifiers, agents, allocators.

> Includes the PDF, the step-by-step guide, a deterministic system prompt, and a worked trading-bot example. ₹499, instant download 👇

Pre-publish checklist

- [ ] Open the ZIP once and confirm all files listed above are present
- [x] ~~Scanned for secrets — none found (this one is clean, unlike crypto/god_ai/video)~~
- [ ] Upload single ZIP to Instamojo as a digital product
- [ ] Set price ₹499, title + description above
- [ ] Add a cover image (the PDF's first page works)
- [ ] Publish → grab the payment link → post the pitch
- [ ] Bank the first rupee. Then raise the price.}

Solana Sniper Listing

A monetization draft with explicit risk and secret-handling caveats.

The following pages reproduce the original source PDF as part of this curated collection.



Instamojo Listing — Solana Sniper: Paper-Trading Study + Full Source (₹2,499)

⚠️ **DO NOT PUBLISH until you redact secrets.** This project currently ships with `.env`, `.env1`, and `gmgn_keys/gmgn_private.pem`. Delete them, rotate the key, add a `.gitignore`, and include a `.env.example` instead. See the checklist at the bottom.

The essentials

Field	Value
Product name	Solana Memecoin Sniper — 6-Month Paper-Trading Study + Hardened Architecture + Full Source
Price	₹2,499
Format	Digital download (single ZIP, secrets redacted)
Category	Developer / trading systems / research
Support	None promised — sold as source + study, as-is
Payout	Instamojo → your bank (KYC done)

The honest positioning (read this first)

You are **not** selling "a bot that makes money." That framing invites furious buyers and legal risk you cannot carry. You are selling something more defensible and, to the right buyer, more valuable: **a serious, six-month engineering study with the full working source.** The honesty *is* the product.

Listing description (paste into Instamojo)

> **A real Solana sniper, paper-traded for 6 months — with an honest risk analysis most "trading bots" will never show you.**

> This is the complete source and research behind a hybrid Solana memecoin sniper: WebSocket/RPC ingestion, an ML signal layer, a real risk-management module, a React + TypeScript dashboard, TDD-hardened risk policy, and systemd services for unattended running.

> **What makes this different: it tells you the truth.** Included is a full `REAL_WORLD_RISK_ANALYSIS` — a 50-run Monte Carlo simulation modeling rugs, MEV sandwiches, latency misses, transaction failures, and slippage. It reports simulated median win rates of 95–98% **and** openly documents what is *not* yet proven: live ML accuracy is unverified, Jito bundle support is incomplete, and no result is a guarantee.

> **What you get:**

> - 🧠 Full source: `solana_hybrid_sniper_ultra` core + ML module + risk manager

> - 💻 React + TypeScript client and server, with a TDD-hardened risk policy and tests

> - 📈 The complete 6-month risk study (Monte Carlo methodology + results)

> - 📄 Architecture + technical presentation docs

> - ⚙️ systemd services + run scripts for unattended operation

> **Who it's for:** Solana devs and quant-curious engineers who want a real, documented sniper architecture to study, extend, and validate themselves — not a black box.

> **Plain terms:** This is source code and research, sold as-is. Paper-tested only; not live-validated. Nothing here is financial advice or a profit guarantee. You run it at your own risk, with your own wallet, after your own validation.

What's inside the ZIP (after redaction)

- `solana_hybrid_sniper_ultra/` — core sniper, `ml/`, `utils/risk_manager.py`
- `client/` — React + Vite dashboard
- `server/` — TypeScript engine, `risk-policy.ts` + tests
- `data/REAL_WORLD_RISK_ANALYSIS.md` — the Monte Carlo study
- Architecture docs (technical presentation, moonshot architecture, convergence framework v3)
- `.trae/specs/` — TDD hardening specs
- `systemd/` + run scripts
- `.env.example` — **replaces** the real `.env`/keys you removed

Short pitch post (for X / Reddit r/solana, r/algotrading)

> Most "trading bots" for sale hide their losses. This one ships the loss analysis.

> Full source of a Solana memecoin sniper + a 6-month paper-trading study: 50-run Monte Carlo modeling rugs, MEV, latency, slippage. Simulated 95–98% win rate — and an honest list of what's NOT yet proven.

> Study it, extend it, validate it yourself. ₹2,499, full source 👇

⚠️ Pre-publish checklist (SECURITY FIRST)

- [ ] **Delete** `.env`, `.env1`, and `gmgn_keys/gmgn_private.pem` from the folder
- [ ] **Rotate** the exposed wallet key + any API keys (assume they are burned)
- [ ] Add a `.gitignore` covering `.env*`, `*.pem`, `gmgn_keys/`, `logs/`
- [ ] Add a clean `.env.example` with variable names only, no values
- [ ] Re-zip the cleaned folder; open the new ZIP and grep for `PRIVATE`, `SECRET`, `.pem` to confirm nothing leaked
- [ ] Write a short `README` with the honest disclaimer above at the top
- [ ] Upload to Instamojo, price ₹2,499, paste title + description
- [ ] Publish → grab link → post the pitch

SafeScreen Roadmap

A B2B roadmap for the safest recurring-revenue path in the archive.

The following pages reproduce the original source PDF as part of this curated collection.



SafeScreen — Roadmap to Real Earning Potential

🎯 **The decision:** SafeScreen is the project to make real. It's the only one with a recurring B2B market and **zero financial/legal risk** — non-negotiable for your situation. The engine already exists; it needs proof and a product wrapper. That's the work only you can do.

Where it actually stands (diagnosed today)

Component	Status
Trained classifier (`toxic_detector.onnx`)	✅ Present, runs
OCR models (PP-OCRv3)	✅ Present
Inference + speed benchmarks	✅ Present (but messy — 6 duplicate entrypoints)
Accuracy / correctness proof	❌ **None. This is the gap.**
Dataset record	❌ `total_images: 0` — no documented training/eval set
Class taxonomy	⚠️ Inconsistent across files
README / docs	❌ None

Translation: it's fast, but unproven. Speed without proven accuracy = not sellable to a business. Fix that one thing and everything downstream unlocks.

The core principle

> Nobody pays monthly for "a model." They pay for **"I can trust this to catch X% of bad content and not falsely flag good content."** A number they can put in a

> compliance doc. Your entire job is to produce that trustworthy number — then wrap it so it's easy to buy.

Phase 0 — Clean the foundation (1 lucid day)

- [ ] Lock ONE final class taxonomy (pick: `violence`, `gore`, `nsfw` — drop `poop/vomit/medical/biological` unless a buyer needs them)
- [ ] Consolidate to **one** inference entrypoint; delete the `_bak` / duplicate scripts
- [ ] Confirm the ONNX model loads and runs one image end-to-end (install `onnxruntime`)
- [ ] Write a short honest README: what it detects, how to run it, current known limits

Phase 1 — VALIDATION (the make-or-break — 3–5 lucid days)

This is the part with your name on it. An agent can build the harness; only you can judge real-world edge cases and source honest test data.

- [ ] Assemble a **held-out test set** you did NOT train on: ~200–500 labeled images per class + a big "safe" set (safe images are where false positives hurt)
- [ ] Build an eval script that outputs: **precision, recall, F1 per class, a confusion matrix, and the false-positive rate on safe images**
- [ ] Run it. Write the numbers down honestly.
- [ ] **Honest gate:** if precision/recall are strong → you have a product. If weak → that's a finding, not a failure: either retrain with more data or narrow the scope to the one class it's good at.
- [ ] Pick a threshold that matches the buyer's need (parental-control apps want high recall; brand-safety wants low false positives)

Phase 2 — Harden (2–3 lucid days)

- [ ] Verify the TFLite/quantized model matches the full model's accuracy (quantization can silently degrade it)
- [ ] Benchmark real latency on a target device; confirm the 30–50ms claim
- [ ] Robust preprocessing + graceful handling of weird inputs (corrupt files, odd sizes, grayscale)
- [ ] Wire in the OCR layer as an optional pass (text-in-image detection = a real differentiator)

Phase 3 — Productize (2–4 days)

- [ ] Choose the form factor (recommended: **hosted API first** — recurring revenue):
  - **Hosted API** → POST an image, get back `{safe: bool, scores: {...}}`. Charge per 1,000 calls or monthly tier. *Best recurring income.*
  - **Model + SDK license** → one-time sale (this is your ₹1,999 Instamojo listing — keep it running in parallel)
  - **Mobile SDK** → for on-device app builders (higher price, more support)
- [ ] Wrap the model in a small FastAPI service (I can scaffold this fully)
- [ ] Build a simple demo page — upload an image, see it flagged. **This is where a tool like base44 fits: the demo/dashboard shell.**
- [ ] Deploy it somewhere cheap (Render/Railway free tier to start)

Phase 4 — Sell it (ongoing)

- [ ] Name the buyer: parental-control apps, kids' platforms, dating apps, community/forum tools, image marketplaces, small app studios needing app-store compliance
- [ ] Pricing draft: Free tier (500 calls) → ₹ / $ per-call tiers → custom for volume
- [ ] Lead with the **accuracy number from Phase 1** + the **privacy angle** ("on-device / your images never leave your server")
- [ ] First outreach: 5–10 indie app devs who post about moderation pain

---

Who does what

The parts only YOU can do	The parts I can do for you
Source honest real-world test images	Build the eval harness + metrics script
Judge edge cases + set thresholds	Scaffold the FastAPI service + demo
Decide the buyer + the price	Write the README + docs
The outreach + the relationships	Clean/consolidate the codebase
Stand behind the accuracy claim	Draft benchmark + pricing pages

Honest risks & kill-criteria

- **If Phase 1 shows the model is weak and can't be fixed with reasonable data** → stop, pivot to selling it as the as-is ₹1,999 artifact, and put energy elsewhere. Validation *protects* you from pouring weeks into something unproven.
- **Don't let this eat the food clock.** Keep the fast track alive (Convergence + reels) while you build this over ~2 weeks of lucid windows.
- **This is a marathon lane, not the first-rupee lane.** First rupee still comes from reels/Convergence. SafeScreen is the *durable* income you build behind it.

Appendix: My Way Out

A crisis-response appendix focused on legal aid, settlement, and immediate stability.

The following pages reproduce the original source PDF as part of this curated collection.



My Way Out — Lawyer Call, Settlement & Bail (Niloy)

🫂 Niloy — this page is yours. Read this line whenever the panic rises: **you are not a criminal. Being unable to repay a loan is not a crime. This has a clear, legal way out, and one free phone call starts it.**

☎️ The one call that starts everything

Free legal aid — NALSA / District Legal Services Authority: dial 15100. It's free. Say you need help with a **loan settlement** and **anticipatory bail**.

🗣️ What to say (read this out — it's calm and complete)

> "My name is Niloy Das. I have a loan I cannot repay — around ₹90 lakhs. I have lost my job and have no assets or income. The lenders have indicated they want to settle. There is a warrant against me, and it is **only** because of this loan — I have not committed any violence or any crime. I want to (1) protect myself from arrest through anticipatory bail, and (2) decide whether to settle or file for insolvency, and handle it properly."

❓ The questions to ask the lawyer

1. Should I go for a **negotiated settlement** or **formal insolvency** — which is better given I have no job and no assets?
2. Can you file for **anticipatory bail** so I'm protected from arrest while this is being sorted?

3. What is the warrant actually for (for example, a cheque-bounce case under Sec. 138, or a complaint filed by the lender)? Is it bailable?
4. How much of the ₹90 lakhs is **legally owed** versus inflated or illegal interest — can the real figure be challenged down?
5. If I settle, how do I get a **"full and final settlement"** in writing so they can never come back?
6. Can the settlement be paid in **installments** instead of one lump sum?

📋 Facts to have ready (fill in the blanks before the call)

- Total owed: **~₹90 lakhs**
- Number of lenders: *[fill in]*
- Type: banks / NBFCs / credit cards / private moneylenders / loan apps — *[fill in which]*
- Did I give any cheques or sign any documents? *[yes/no — details]*
- Settlement amount discussed: *[if known]*
- Job / income: **none currently**
- Assets: *[list anything, or "none"]*
- The warrant: **loan-related only; no violence, no crime committed**

📞 Keep these numbers

- **NALSA free legal aid: 15100**
- **KIRAN mental-health helpline (24/7, free): 1800-599-0019**
- **Cyber-crime / loan-app harassment: 1930** or **cybercrime.gov.in**
- **Emergency: 112**

🫂 If the weight gets too heavy

This is a lot for one person to carry. If the dark thoughts come back, please call **KIRAN (1800-599-0019)** or **Tele-MANAS (14416)**. You don't have to solve everything today — getting through the next hour is enough. You reached out this morning instead of disappearing. Keep doing that.

The real headline: a debt the lenders want to settle, no assets to seize, and a warrant that is only about money — this is a resolvable situation with a known legal path. One call to 15100 starts it.